
CS246: Database Management Systems Lab

Week # 01 (1 Questions, 130 Marks)

Lab session: AL1

Held on: 12-Jan-2026 (Mon)

Lab Timings: 14:00 to 17:00 Hours Pages: 4

Submission time: 17:00 Hrs, 12-Jan-2026

Instructors Dr. V. Vijaya Saradhi

Department of CSE, IIT Guwahati

Question 1: (130 points)

Given two files namely `s1.csv` and `s2.csv` whose file format is described below:

1. Input files `s1.csv` and `s2.csv` have four columns as described

1 st column	<code>sid</code>	Sailor id	<code>int</code>
2 nd column	<code>sname</code>	Sailor name	<code>char(20)</code>
3 rd column	<code>rating</code>	Sailor rating	<code>int</code>
4 th column	<code>age</code>	Sailor age	<code>int</code>

Note 1: Treat a sailor record as uniquely identified by `sid`. If the same `sid` appears in both files, assume the rows are identical;

Note 2: There are no duplicates in `s1.csv` or `s2.csv`.

Note 3: Data starts from first line with no header information in both `s1.csv` and `s2.csv`.

Perform the following:

Task 01 (4 marks) Declare a structure `sailor` holding one row of input file consisting of four members.

1. First member is an `int` storing `sid`.
2. Second member is a one-dimensional `char` array storing `sname`.
3. Third member is an `int` storing `rating`.
4. Fourth member is an `int` storing `age`.

Task 02 (3 marks) Declare a structure `node` that represents a node of a binary search tree consists of three members.

1. 1st member is of type structure `sailor`
2. 2nd member is a pointer to the structure `node` denoting left sub-tree
3. 3rd member is a pointer to the structure `node` denoting right sub-tree

Task 03 (40 marks) Implement the binary search tree `insert` function considering the comparison key to be `sid` whose specifications are:

1. **Input arguments:** Two input arguments
 - (a) 1st input argument: a pointer to **node** structure representing root of binary search tree
 - (b) 2nd input argument: a structure of type **sailor**
2. **Return argument** a pointer to **node** structure representing updated root of binary search tree
3. **Functionality** Perform sailor record insertion into appropriate position of binary search tree.

Task 04 (10 marks) Implement the **find** function considering the comparison key to be **sid**. Specifications of **find** function are:

1. **Input arguments:** Two input arguments
 - (a) 1st input argument: a pointer to **node** structure representing root of binary search tree
 - (b) 2nd input argument: an integer representing **sid**
2. **Return argument:** **bool**
3. **Functionality:** If **sid** present in the given binary search tree then return true; else return false;

Task 05 (10 marks) Implement the pre-order tree traversal whose specifications are:

1. **Input argument** a pointer to **node** structure representing BST
2. **Return argument** **void**
3. **Functionality** Traverse the tree and print every node (containing four columns) of the tree.

Task 06 (10 marks) Read every line from **s1.csv** and insert into a first binary search tree B1.

Task 07 (10 marks) Read every line from **s2.csv** and insert into a second binary search tree B2.

Task 08 (20 marks) Computing Union of Two Binary Search Trees Write a function **union** whose specification is:

1. **Input arguments**
 - (a) 1st argument: a pointer to **node** structure representing root of BST B1
 - (b) 2nd argument: a pointer to **node** structure representing root of BST B2
2. **Returns:** a pointer to **node** structure representing root of BST B3
3. **Functionality:** The function **union** constructs a third binary search tree B3 containing the union of elements from B1 and B2.

The union operation must be implemented incrementally during traversal, without converting the trees into arrays or lists.

Specifically:

- (a) Perform a pre-order traversal of B1; for each visited node, insert its sailor record into B3 if it does not already exist.
- (b) Perform a pre-order traversal of B2; for each visited node, insert its sailor record into B3 if it does not already exist.
- (c) Two nodes are considered identical if their search key (sid) of sailor record is equal.
- (d) B3 must be a valid binary search tree, but its shape need not match B1 or B2.

Important Note: Using **inorder** traversal followed by sorting/merging will be considered incorrect.

Example:

s1.csv			
sid	sname	rating	age
22	Dustin	7	45

s2.csv			
sid	sname	rating	age
31	Lubber	8	55

s1.csv $\cup$ s2.csv			
sid	sname	rating	age
22	Dustin	7	45
31	Lubber	8	55

Definition: $B1 \cup B2 = \{ r \mid r \in B1 \text{ OR } r \in B2 \}$

Task 09 (20 marks) Computing Intersection of Two Binary Search Trees Write a function **intersection** whose specification is:

1. **Input arguments**

- (a) 1st argument: a pointer to **node** structure representing BST B1
- (b) 2nd argument: a pointer to **node** structure representing BST B2

2. **Returns:** a pointer to **node** structure representing BST B3

3. **Functionality:** construct a third binary search tree B3 containing the intersection of elements from two given binary search trees B1 and B2.

- (a) Perform a pre-order traversal of B1; for each visited node, insert its sailor's record into B3 (1) if the node exists in B2 as well and (2) if the node is not present in B3.

- (b) Perform a pre-order traversal of B2; for each visited node, insert its sailor's record into B3 (1) if the node exists in B2 as well and (2) if it does not already exist in B3.
- (c) Two nodes are considered identical if their search key (sid) is equal.
- (d) B3 must be a valid binary search tree, but its shape need not match B1 or B2.
- (e) Each key must appear only once in B3, even if it exists multiple times across B1 and B2.
- (f) B3 must satisfy the binary search tree property, but its structure need not match either B1 or B2.
- (g) No intermediate storage (arrays, vectors, lists, or sorted sequences) may be used for computing the intersection.

The intersection operation must be implemented incrementally during tree traversal, without converting the trees into arrays, lists, or any other intermediate data structures.

Example:

s1.csv			
sid	sname	rating	age
22	Dustin	7	45

s2.csv			
sid	sname	rating	age
22	Dustin	7	45
31	Lubber	8	55

s1.csv $\cap$ s2.csv			
sid	sname	rating	age
22	Dustin	7	45

Definition: $B1 \cap B2 = \{ r \mid r \in B1 \text{ AND } r \in B2 \}$

Task 11 (3 marks) Free the memory corresponding to B1, B2, B3 and B4 whose specifications are:

1. **Input argument** a pointer to **node** structure representing BST
2. **Return argument** void
3. **Functionality** Free every node of the binary search tree